

Rechnernetze Praktikum 3

Author: Richard Salim, Wildan Syufie

In diesem Protokoll schauen wir die Anwendung zwischen Client und einem multiclientfähigen Server unter Verwendung von TCP. In diesem Programm gibt es ein paar Client Kommando, die hier implementiert werden. Die Anwendung soll protokollunabhängig, also sowohl für IPv4 als auch IPv6, lauffähig sein.

Am Anfang müssen wir erstmal unsere Netzwerk Client zu unserem Netzwerk Server verbinden. Da wir im Labor die Lab 26 benutzt haben, müssen wir die SSH-Verbindung mit Netzwerk lab 33 zusammenkonfigurieren.

```
networker@lab26:~$ ssh networker@lab33
The authenticity of host 'lab33 (141.22.27.112)' can't be established.
ECDSA key fingerprint is SHA256:6Z//DqaXxCtX3lYYXZ6Gtbq4647s/+P8gQjyS35kDOM.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'lab33,141.22.27.112' (ECDSA) to the list of known hosts.
networker@lab33's password:
Welcome to Ubuntu 20.04.3 LTS (GNU/Linux 5.4.0-110-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

434 updates can be applied immediately.
2 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Last login: Thu Sep 13 14:05:36 2018 from 141.22.26.159
```

Abbildung 1 : SSH Connection

Wenn die Verbindung schon angeschlossen wird, dann kann die „ifconfig“ Kommando angerufen werden, um die IPv4 und IPv6 von Netzwerk lab 33 anzuschauen. Diese IP-Adressen benutzt der Client als DNS-Name, um die Verbindung mit dem Server zu bauen.

```
eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.18.136 netmask 255.255.255.0 broadcast 192.168.18.255
    inet6 fe80::21b:21ff:fe40:e6b4 prefixlen 64 scopeid 0x20<link>
    inet6 fd32:6de0:1f69:18:21b:21ff:fe40:e6b4 prefixlen 64 scopeid 0x0<global>
    ether 00:1b:21:40:e6:b4 txqueuelen 1000 (Ethernet)
    RX packets 30 bytes 2186 (2.1 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 67 bytes 10442 (10.4 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device memory 0xf7c20000-f7c3ffff

eth2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.16.1.14 netmask 255.255.255.0 broadcast 172.16.1.255
    inet6 fd32:6de0:1f69:16:21b:21ff:fe40:e6b5 prefixlen 64 scopeid 0x0<global>
    inet6 fe80::21b:21ff:fe40:e6b5 prefixlen 64 scopeid 0x20<link>
    ether 00:1b:21:40:e6:b5 txqueuelen 1000 (Ethernet)
    RX packets 4 bytes 598 (598.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 61 bytes 9413 (9.4 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device memory 0xf7c00000-f7c1ffff
```

Abbildung 2 : Ifconfig command

Wir nehmen als default Port in Server die 0. Dies bedeutet, dass der Socket einen Port vom Betriebssystem zugewiesen bekommt. Mit dem Kommando „netstat -lnt“ können wir anschauen, welche spezifischen Port ist verfügbar zu wählen.

```
networker@lab33:~/Desktop/rn-exercise3-template/template/src/Server$ ./s
erver 0
Server running
data size 84
sent: 84 bytes
```

Abbildung 3: Server running with default Port 0

```
networker@lab33:~$ netstat -lnt
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:631           0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:9400            0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:36437         0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN
tcp6       0      0 :::22                   :::*                     LISTEN
tcp6       0      0 :::1:631                 :::*                     LISTEN
tcp6       0      0 :::12865                 :::*                     LISTEN
tcp6       0      0 :::54923                 :::*                     LISTEN
tcp6       0      0 :::8811                  :::*                     LISTEN
```

Abbildung 4 : show all the Address that Server listening using "netstat -lnt" command

Nun kann der Client die gegebenen IP4 oder 1P6-Adressen von eth1/eth2 und die gegebenen Ports benutzen. Ab hier kann die Verbindung zwischen Client und Server abgebaut werden.

In diesem Programm gibt es ein paar Client Kommando, die hier implementiert werden bzw, abgeführt werden kann.

1. List = Hier zeigt die Liste aller verbundenen Clients.

```
List
::ffff:172.16.1.8: 57032
```

Abbildung 5 : List all connected Clients

2. Files = Mit diesem Kommando wird die Liste aller Dateien im Server-Verzeichnis gezeigt.

```
Files
filename: server.cc Last modified: Thu May 19 16:15:43 2022
size: 11329 bytes
filename: Makefile Last modified: Thu May 19 16:17:01 2022
size: 93 bytes
filename: test1.txt Last modified: Tue May 24 10:37:15 2022
size: 4 bytes
filename: test.txt Last modified: Tue May 24 10:37:05 2022
size: 550 bytes
filename: big text.txt Last modified: Tue May 24 10:37:36 2022
size: 2167737 bytes
filename: server Last modified: Tue May 24 11:25:11 2022
size: 202536 bytes
filename: test2.txt Last modified: Tue May 24 10:37:23 2022
size: 84 bytes
```

Abbildung 6 : Show all Files in Server

3. Get = Mit diesem Kommando kann der Client die Datei von Server einnehmen und wird auch in Client gespeichert. Wir haben hier mit 2 Typ von Dateien versuchen, eine ist groß Datei und die andere ist klein Datei. In unsere Abbildung kann man sehen, dass die beiden Dateien von Server perfekt abgeholt werden. Der Unterschied zwischen die beiden Dateien ist nur die Laufzeit, die größere Datei braucht mehr Zeit als die kleinere Datei.

```
networker@lab26:~/Desktop/rn-exercise3-template/template/src/Client$ ./client fd32:6de0:1f69:16:21b:21ff:fe40:e6b5 54923
Connecting to remote host fd32:6de0:1f69:16:21b:21ff:fe40:e6b5
Get test2.txt

File size: 84
Last modified : 4461719-03-28 14:02:48
Das ist ein Test
Das ist ein Test
Das ist ein Test
Das ist ein Test
Das ist ein Test
Transmission Time: 0.022000 ms
List
fd32:6de0:1f69:16:21b:21ff:fe40:e7fd: 49898
█
```

Abbildung 7 : Small Data with Get command in Client view

```
at augue eget arcu dictum. Ac tortor dignissim convallis aenean et tortor at risus. Lorem ipsum dolor sit
erdum posuere lorem ipsum dolor. Proin libero nunc consequat interdum varius sit amet mattis. Vel quam elem
iquet bibendum. Libero volutpat sed cras ornare arcu dui vivamus arcu felis. Cras sed felis eget velit alic
non diam phasellus vestibulum lorem sed. Bibendum est ultricies integer quis.
Transmission Time: 5.541000 ms
█
```

Abbildung 8 : Big Data with Get command in Client view

```
data size 84
sent: 84 bytes
█
```

Abbildung 9 : Small data with Get command in Server view

```
data size 2167737
sent: 2167737 bytes
█
```

Abbildung 10 : Big data with Get command in Server view

4. Put = Mit diesem Kommando kann der Client die Datei nach Server absenden und wird auch in Server gespeichert. Wir haben hier auch mit 2 Typ von Dateien versuchen, eine ist groß Datei und die andere ist klein Datei. In unsere Abbildung kann man sehen, dass die beiden Dateien nach Server perfekt geschickt werden. Der Unterschied zwischen die beiden Dateien ist nur die Laufzeit, die größere Datei braucht mehr Zeit als die kleinere Datei.

```
Put test.txt
sent: 550 bytes
Server = lab33.mailcluster.haw-hamburg.de
Server IP = ::
Current Date and time = : 2022-05-24 11:51:50
```

Abbildung 11 : Small data with Put command in Client view

```
Put big text.txt
sent: 2167737 bytes
Server = lab33.mailcluster.haw-hamburg.de
Server IP = ::
Current Date and time = : 2022-05-24 12:01:37
```

Abbildung 12 : Big data with Put command in Client view

```
waiting rest data...
received...
received 550 bytes, datasize: 550 bytes
File saved successfully
```

Abbildung 13 : Small data with Put command in Server view

```
OUTPUT  TERMINAL  DEBUG CONSOLE  PROBLEMS
waiting rest data...
received...
waiting rest data...
received...
waiting rest data...
received...
waiting rest data...
received...
waiting rest data...
received...
waiting rest data...
received...
waiting rest data...
received...
waiting rest data...
received...
waiting rest data...
received...
received 2167737 bytes, datasize: 2167737 bytes
File saved successfully
```

Abbildung 14 : Big data with Put command in Server view

5. Quit = Zuletzt haben wir die Quit Kommando, dieses Kommando benutzen wir, um die Verbindung zwischen Client und Server zu beenden.

```
Quit
networker@lab26:~/Desktop/rn-exercise3-template/template/src/Client$
```

Abbildung 15 : Quit command in Client view

Zum Schluss müssen wir alle Dateien in Server und Client mit dem CLI-Tool „diff“ überprüfen, dass die Dateien, die geschickt bzw. genommen haben, gleich sind. Hier haben wir 2 Test gemacht, eine

mit dem gleichen Text-Dateien, und die andere mit unterschiedlichen Text-Dateien zu vergleichen.
Diese Tests brauchen wir zu überprüfen, dass die Dateien genau richtig die Gleiche sind.

```
networker@lab26:~/Desktop/rn-exercise3-template/template/src$ diff Server/test.txt Client/test.txt
networker@lab26:~/Desktop/rn-exercise3-template/template/src$ diff Server/test1.txt Client/test1.txt
networker@lab26:~/Desktop/rn-exercise3-template/template/src$ diff Server/test2.txt Client/test2.txt
networker@lab26:~/Desktop/rn-exercise3-template/template/src$ diff Server/big_text.txt Client/big_text.txt
```

Abbildung 16 : CLI-Tool "Diff" with the same Data

```
networker@lab26:~/Desktop/rn-exercise3-template/template/src$ diff Server/test1.txt Client/test2.txt
1c1,5
< test
\ No newline at end of file
---
> Das ist ein Test
> Das ist ein Test
> Das ist ein Test
> Das ist ein Test
> Das ist ein Test
> Das ist ein Test
\ No newline at end of file
```

Abbildung 17 : CLI-Tool "diff" with difference Data